

Sera is a recruitment OS built as three separate portals -- Candidate, Recruiter, and YZI Admin -- that never talk to each other directly. YZI Admin is the only party that can see both a candidate's real contact details and a recruiter's requirement. This document maps the UI flow already built (Vite/React/TS, mock data) to the backend that still needs to exist behind it.

1. Three-portal architecture



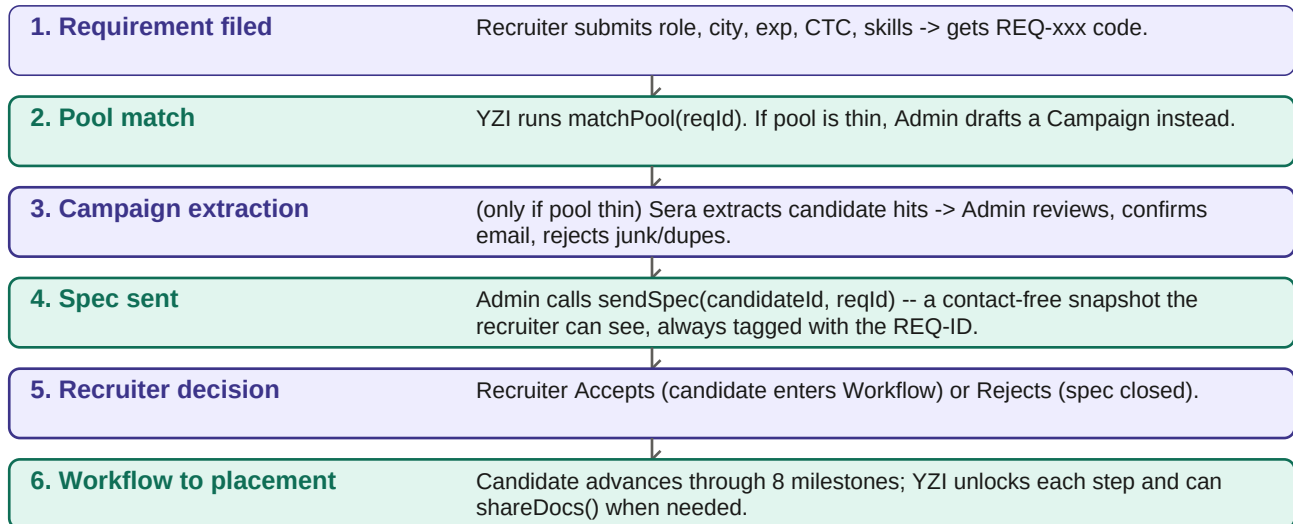
No API route should ever let a recruiter session read a candidate's phone, email, or address -- and vice versa.

2. What each portal is allowed to see

Candidate	Own profile, own document uploads, own milestone status, YZI-only inbox. Never sees a recruiter, a requirement, or another candidate.
Recruiter	Requirements they filed (REQ-xxx), the candidate pool matched to those requirements as anonymised specs (name/stack/city/years -- no contact fields), and workflow progress. Accept/Reject moves a spec, it never reveals contact info.
YZI Admin	Everything: full candidate identity, all requirements, campaign extraction, pool kanban, approvals queue, and the only account that can send a spec, share documents, confirm an email, or reveal identity. Every one of those actions must write an insert-only audit_events row.

3. Requirement-to-placement pipeline

This is the sequence a single hire follows end to end. Pool matching is tried first; a campaign is only started when the house pool is thin for that requirement.



4. Candidate milestone track (workflow_step)



Note: schema's workflow_step enum only has 6 values (profile, meeting, docs, interview, offer, placed) -- UI shows 8. Reconcile before building the API.

Schema file exists but nothing is live yet (no Supabase auth wired up this step). Tables below are the source of truth for what the backend needs to implement. Key privacy rule: the specs table has no contact columns at all -- that's what makes the recruiter view safe by construction.

orgs	id, name, type (yzi recruiter)
users	id, org_id, role (candidate recruiter admin), name, email, phone
candidates	id, user_id, name fields, city, address, expertise[], years, ctc fields, cv_url, account_status, source, lock, locked_spec_id
requirements	id, org_id, recruiter_user_id, code (REQ-xxx), role, city, skills[], status (new pool_ok pool_thin campaign filled)
campaigns	id, org_id, req_id (nullable -- YZI can run in-house), way, status (draft running ended), target/extracted/shortlisted counts
hits	id, campaign_id, candidate_id, extracted jsonb, tag (cleared no_email dupe junk), score
invite_tokens	id, token, campaign_id, req_id, email, used_at -- candidate signup reads this to set source=campaign_link
specs	id, req_id, recruiter_org_id, recruiter_user_id, candidate_id, snapshot jsonb, status (pool rejected workflow) -- NO contact columns, by design
workflows	id, spec_id, step (workflow_step enum)
messages	id, admin_id, side (recruiter candidate), recruiter_id, candidate_id, req_id, body, created_at -- YZI is always one side of every thread
documents	id, candidate_id, name, url, shared_spec_id
identity_requests	id, candidate_id, field, from_value, to_value, reason, status (pending approved rejected)
audit_events	id, actor_id, action, subject_type, subject_id, meta jsonb, ip, created_at -- INSERT ONLY, update/delete blocked by DB rule

src/api/stubs.ts currently returns hardcoded fake data for five functions -- these are the real backend endpoints your coder needs to build. Frontend already calls these by name, so keeping the same function signatures means no UI changes are needed once they're live.

matchPool(requirementId)	Returns candidates already matched from the house pool for a REQ. Should query candidates joined against requirement skills/city, respecting lock status.
startCampaign(brief)	Creates a draft campaign row (role, city, skills) when the pool is thin for a requirement. Real version: insert into campaigns with status=draft.
sendSpec(candidateId, requirementId)	Writes a specs row with a contact-free snapshot jsonb and status=pool, then an audit_events row (action=send_spec, meta must include recruiter_name, recruiter_org, req_code per the schema comment).
shareDocs(candidateId)	Marks documents visible to a spec (shared_spec_id) and logs an audit_events row. Admin-only action.
confirmEmail(candidateId)	Confirms a campaign hit's email during campaign review, moving it from extracted to a real candidate/spec record.

5. Build priorities for the backend

- Auth + roles first: candidate / recruiter / admin sessions mapped to users.role. No live Supabase auth exists yet -- this blocks everything else.
- Enforce contact isolation at the query layer, not just in the UI -- recruiter-facing endpoints must select specs.snapshot, never candidates.email/phone/address directly.
- Make audit_events writes automatic (a DB trigger or a shared server-side wrapper) around sendSpec, shareDocs, and identity reveals, so no endpoint can skip logging.
- Reconcile the workflow_step enum (6 values in schema) against the 8-stage UI milestone track on page 2 before wiring the workflow endpoints.
- Real Naukri/LinkedIn scraping is explicitly out of scope per the project brief -- campaign 'extraction' should stay mocked or use a licensed data source, not a live scraper.